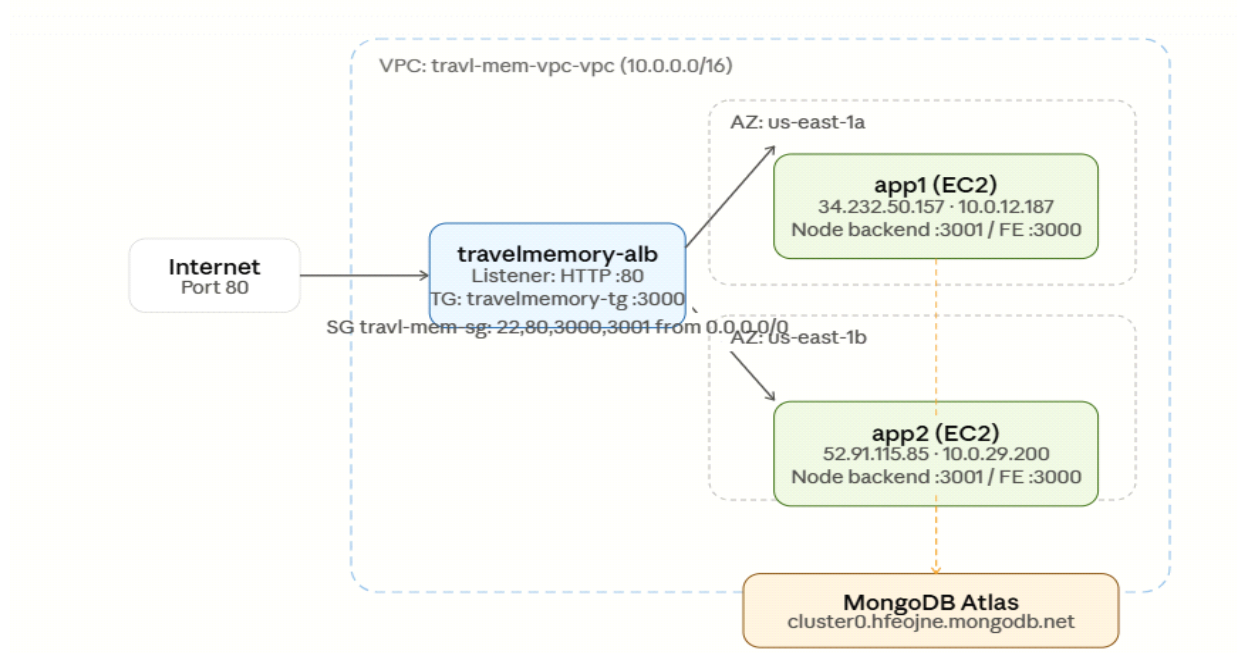
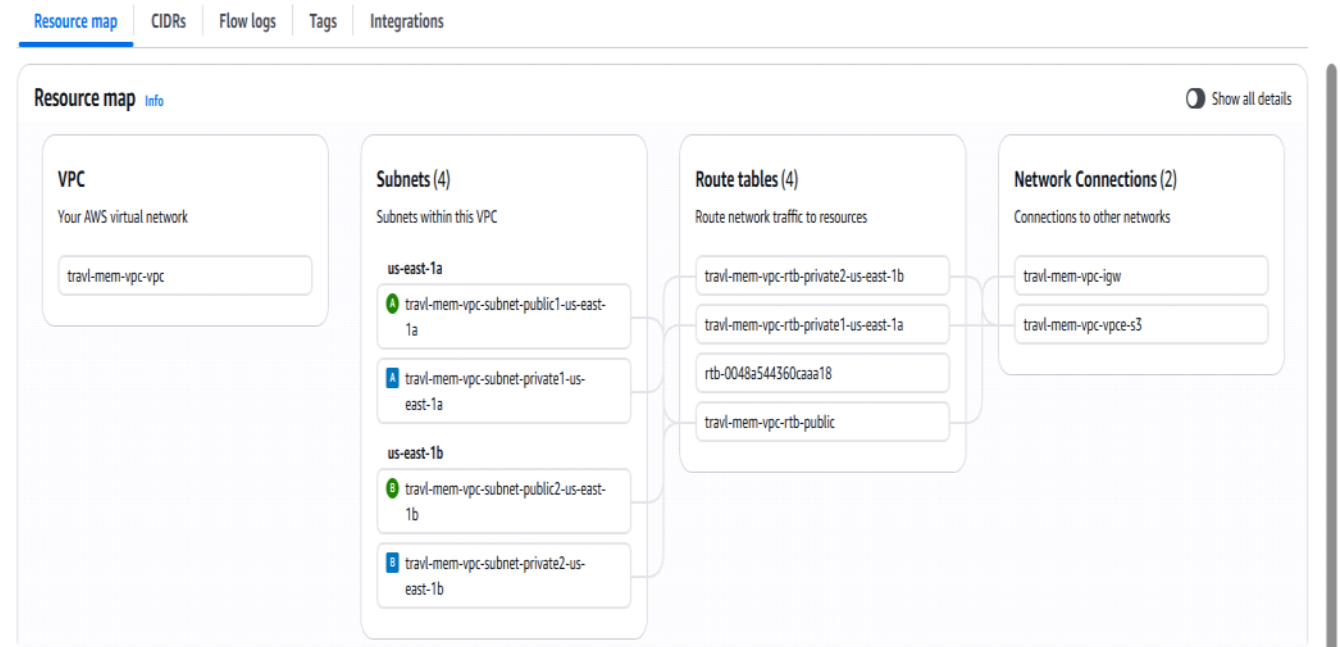


Travel-Memory-Multi Node with ALB

29 June 2026 03:13 PM



#VPC creation



#Subnet

VPC dashboard < Subnets (10) Info

Find subnets by attribute or tag

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR
-	subnet-05f74d7d84264857b	Available	vpc-0c7e03346b9b0b787	Off	172.31.32.0/20	-
travl-mem-vpc-subnet-public2-us-east-1b	subnet-06a35e06ce20ad88f	Available	vpc-0f6b9f93a5304573b travl-mem-vpc-vpc	Off	10.0.16.0/20	-
-	subnet-0b210345ed7c1e1be	Available	vpc-0c7e03346b9b0b787	Off	172.31.80.0/20	-
-	subnet-029bf63c0a5363a3b	Available	vpc-0c7e03346b9b0b787	Off	172.31.64.0/20	-
travl-mem-vpc-subnet-public1-us-east-1a	subnet-04953f1ff97206d	Available	vpc-0f6b9f93a5304573b travl-mem-vpc-vpc	Off	10.0.0.0/20	-
-	subnet-041032c66b097e8e7	Available	vpc-0c7e03346b9b0b787	Off	172.31.0.0/20	-
-	subnet-061cf56254ad9815b	Available	vpc-0c7e03346b9b0b787	Off	172.31.16.0/20	-
travl-mem-vpc-subnet-private1-us-east-1a	subnet-0389c4219f229877	Available	vpc-0f6b9f93a5304573b travl-mem-vpc-vpc	Off	10.0.128.0/20	-
travl-mem-vpc-subnet-private2-us-east-1b	subnet-091531af20e12c5ff	Available	vpc-0f6b9f93a5304573b travl-mem-vpc-vpc	Off	10.0.144.0/20	-
-	subnet-0797e905a263436d3	Available	vpc-0c7e03346b9b0b787	Off	172.31.48.0/20	-

Select a subnet

#secutiry group create and open port inbound access

VPC dashboard < Security Groups > sg-0d17f8c3465821ea2 - travl-mem-sg

Security group (sg-0d17f8c3465821ea2 | travl-mem-sg) was created successfully

sg-0d17f8c3465821ea2 - travl-mem-sg

Details

Security group name travl-mem-sg	Security group ID sg-0d17f8c3465821ea2	Description travl-mem-sg	VPC ID vpc-0f6b9f93a5304573b
Owner 675134942081	Inbound rules count 4 Permission entries	Outbound rules count 1 Permission entry	

Inbound rules (4)

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-00cf61b0033835ebf	IPv4	SSH	TCP	22	0.0.0.0/0	-
-	sgr-0c5ebaf0356512068	IPv4	HTTP	TCP	80	0.0.0.0/0	-
-	sgr-067999362efc28f46	IPv4	Custom TCP	TCP	3000	0.0.0.0/0	-
-	sgr-0afa39d437e9ecea	IPv4	Custom TCP	TCP	3001	0.0.0.0/0	-

EC2 creating - app1

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags [Info](#)

Name

app1

[Add additional tags](#)

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Q Search our full catalog including 1000s of application and OS images

Recents

[Quick Start](#)



[Browse more AMIs](#)
Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Ubuntu Server 26.04 LTS (HVM), SSD Volume Type
ami-0b6d9d3d33ba97d99 (64-bit (x86)) / ami-0bc7f2dbdccc6b5303 (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

Amazon Machine Image (AMI)

Ubuntu Server 26.04 LTS (HVM), SSD Volume Type
ami-0b6d9d3d33ba97d99 (64-bit (x86)) / ami-0bc7f2dbdccc6b5303 (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

Description

Ubuntu Server 26.04 LTS (HVM), EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).

Canonical, Ubuntu, 26.04, amd64 resolute image

Architecture

64-bit (x86)

AMI ID

ami-0b6d9d3d33ba97d99

Publish Date

2026-06-04

Username

ubuntu



Verified provider

▼ Instance type [Info](#) | [Get advice](#)

Instance type

t3.micro

Family: t3 2 vCPU 1 GiB Memory Current generation: true On-Demand Linux base pricing: 0.0104 USD per Hour
On-Demand Windows base pricing: 0.0196 USD per Hour On-Demand Ubuntu Pro base pricing: 0.0139 USD per Hour
On-Demand SUSE base pricing: 0.0104 USD per Hour On-Demand RHEL base pricing: 0.0392 USD per Hour

Free tier eligible

☐ All generations

[Compare instance types](#)

[Additional costs apply for AMIs with pre-installed software](#)

▼ Instance type [Info](#) | [Get advice](#)

Instance type

t3.micro

Family: t3 2 vCPU 1 GiB Memory Current generation: true On-Demand Linux base pricing: 0.0104 USD per Hour
On-Demand Windows base pricing: 0.0196 USD per Hour On-Demand Ubuntu Pro base pricing: 0.0139 USD per Hour
On-Demand SUSE base pricing: 0.0104 USD per Hour On-Demand RHEL base pricing: 0.0392 USD per Hour

Free tier eligible

☐ All generations

[Compare instance types](#)

[Additional costs apply for AMIs with pre-installed software](#)

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*

trvl-mem-tst

[Create new key pair](#)

▼ Network settings [Info](#)

VPC - *required* [Info](#)

vpc-0f6b9f93a5304573b (travl-mem-vpc-vpc)
10.0.0.0/16



Subnet [Info](#)

subnet-04953f1fbcf97206d travl-mem-vpc-subnet-public1-us-east-1a
VPC: vpc-0f6b9f93a5304573b Owner: 675134942081 Availability Zone: us-east-1a (use1-az1)
Zone type: Availability Zone IP addresses available: 4091 CIDR: 10.0.0.0/20

[Create new subnet](#)

Auto-assign public IP [Info](#)

Enable

Firewall (security groups) [Info](#)

EC2 > Instances > Launch an instance

Firewall (security groups) Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group

Select existing security group

Common security groups Info

Select security groups

travl-mem-sg sg-0d17f8c3465821ea2 X
VPC: vpc-0f6b9f93a5304573b

Compare security group rules

Security groups that you add or remove here will be added to or removed from all your network interfaces.

Advanced network configuration

Configure storage Info

Advanced

1x 8 GiB gp3 Root volume, 3000 IOPS, Not encrypted

Add new volume

The selected AMI contains instance store volumes, however the instance does not allow any instance store volumes. None of the instance store volumes from the AMI will be accessible from the instance

Click refresh to view backup information
The tags that you assign determine whether the instance will be backed up by any Data Lifecycle Manager policies.

File systems

S3 Files - new

EFS

F5x

None

▼ S

Numt

1

Softw

Canor

ami-0c

Virtua

t3.mik

Firew

travl-r

Storag

1 volu

Can

EC2-app2 creation

EC2 > Instances > Launch an instance

Launch an instance Info

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags Info

Name

app2

Add additional tags

▼ Application and OS Images (Amazon Machine Image) Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search our full catalog including 1000s of application and OS images

Recents

Quick Start

Amazon Linux

macOS

Ubuntu

Windows

Red Hat

SUSE Linux

Debian

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Ubuntu Server 26.04 LTS (HVM), SSD Volume Type
ami-0b6d9d3d33ba97d99 (64-bit (x86)) / ami-0bc7f2dbdcc6b5303 (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

Travel-memory-ec2-LB Page 4

aws

Search

[Alt+S]

EC2 > Instances > Launch an instance

Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Ubuntu Server 26.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).

Canonical, Ubuntu, 26.04, amd64 resolute image

Architecture

AMI ID

Publish Date

Username

Verified provider

64-bit (x86) ami-0b6d9d3d33ba97d99 2026-06-04 ubuntu

▼ Instance type

Info | Get advice

Instance type

t3.micro

Family: t3 2 vCPU 1 GiB Memory Current generation: true On-Demand Linux base pricing: 0.0104 USD per Hour On-Demand Windows base pricing: 0.0196 USD per Hour On-Demand Ubuntu Pro base pricing: 0.0139 USD per Hour On-Demand SUSE base pricing: 0.0104 USD per Hour On-Demand RHEL base pricing: 0.0392 USD per Hour

Free tier eligible

All generations

Compare instance types

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login)

Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

trvl-mem-tst

Create new key pair

EC2 > Instances > Launch an instance

▼ Network settings

Info

VPC - required

Info

vpc-0f6b9f93a5304573b (travl-mem-vpc-vpc)

10.0.0.0/16

Subnet

Info

subnet-06a35e06ce20ad88f

travl-mem-vpc-subnet-public2-us-east-1b

VPC: vpc-0f6b9f93a5304573b Owner: 675134942081 Availability Zone: us-east-1b (use1-az2) Zone type: Availability Zone IP addresses available: 4091 CIDR: 10.0.16.0/20

Create new subnet

Auto-assign public IP

Info

Enable

Firewall (security groups)

Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group

Select existing security group

Common security groups

Info

Select security groups

travl-mem-sg sg-0d17f8c3465821ea2

VPC: vpc-0f6b9f93a5304573b

Compare security group rules

Security groups that you add or remove here will be added to or removed from all your network interfaces.

► Advanced network configuration

EC2 > Instances > Launch an instance

Common security groups

Info

Select security groups

travl-mem-sg sg-0d17f8c3465821ea2

VPC: vpc-0f6b9f93a5304573b

Compare security group rules

Security groups that you add or remove here will be added to or removed from all your network interfaces.

► Advanced network configuration

▼ Configure storage

Info

Advanced

1x 8 GiB gp3

Root volume, 3000 IOPS, Not encrypted

Add new volume

The selected AMI contains instance store volumes, however the instance does not allow any instance store volumes. None of the instance store volumes from the AMI will be accessible from the instance

Click refresh to view backup information

The tags that you assign determine whether the instance will be backed up by any Data Lifecycle Manager policies.

File systems

S3 Files - new

FSx

EFS

None

► Advanced details

Info

▼ Summary

Info

Number of instances

1

Software Image (AMI)

Canonical, Ubuntu, 26.04, amd6...read more

ami-0b6d9d3d33ba97d99

Virtual server type (instance type)

t3.micro

Firewall (security group)

travl-mem-sg

Storage (volumes)

1 volume(s) - 8 GiB

Cancel

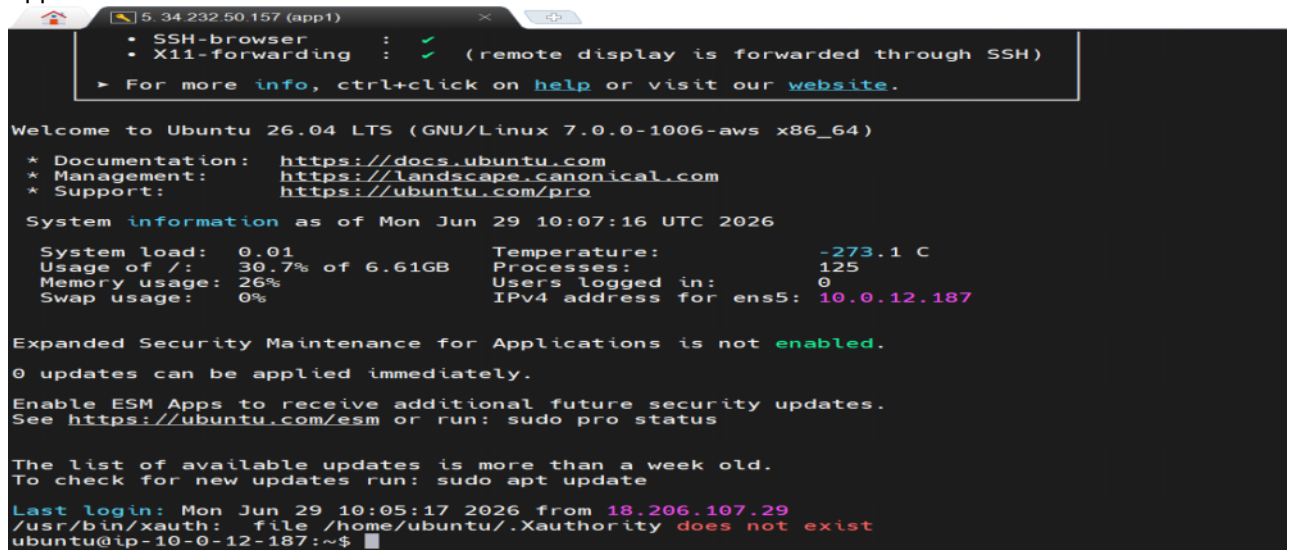
Launch instance

Preview code

Travel-memory-ec2-LB Page 5

Check the connectivity both ec2 are able to connect through ssh

App1 :



```
5. 34.232.50.157 (app1)
• SSH-browser      : ✓ (remote display is forwarded through SSH)
• X11-forwarding   : ✓
► For more info, ctrl+click on help or visit our website.

Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Jun 29 10:07:16 UTC 2026

System load:  0.01          Temperature:    -273.1 C
Usage of /:   30.7% of 6.61GB Processes:     125
Memory usage: 26%          Users logged in: 0
Swap usage:  0%            IPv4 address for ens5: 10.0.12.187

Expanded Security Maintenance for Applications is not enabled.

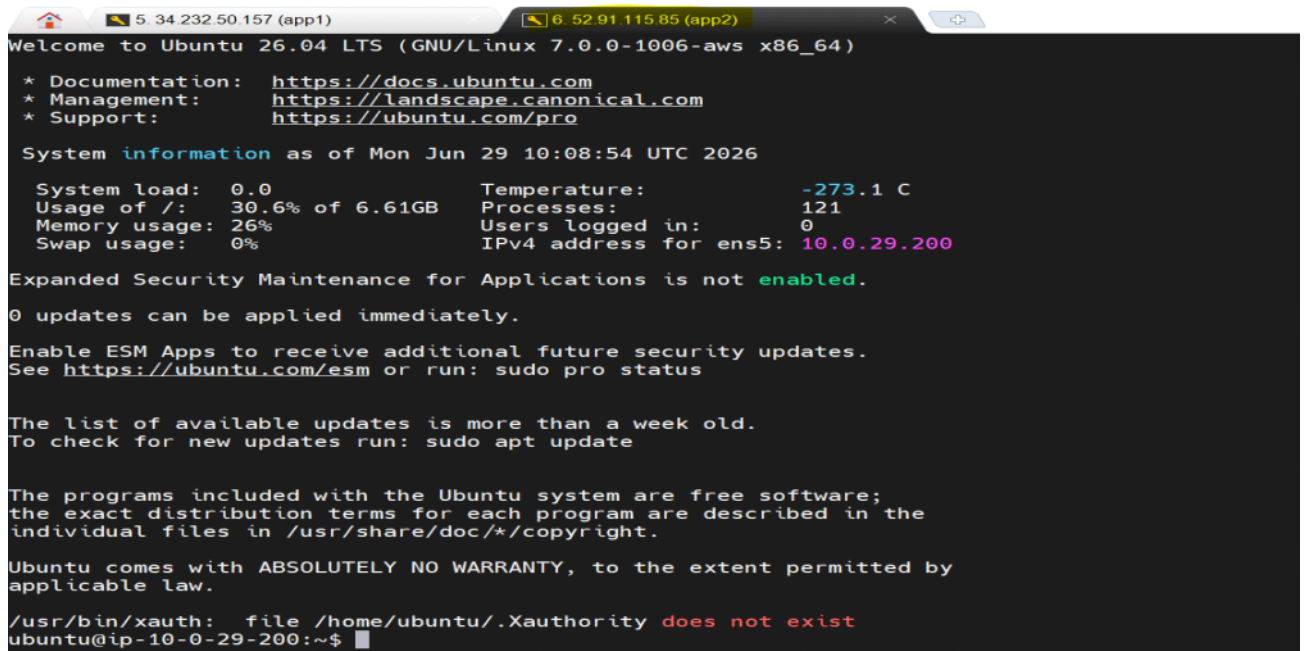
0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Mon Jun 29 10:05:17 2026 from 18.206.107.29
/usr/bin/xauth: file /home/ubuntu/.Xauthority does not exist
ubuntu@ip-10-0-12-187:~$
```

App2 :



```
5. 34.232.50.157 (app1)  6. 52.91.115.85 (app2)
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Jun 29 10:08:54 UTC 2026

System load:  0.0          Temperature:    -273.1 C
Usage of /:   30.6% of 6.61GB Processes:     121
Memory usage: 26%          Users logged in: 0
Swap usage:  0%            IPv4 address for ens5: 10.0.29.200

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

/usr/bin/xauth: file /home/ubuntu/.Xauthority does not exist
ubuntu@ip-10-0-29-200:~$
```

Introduction to Updating EC2 Instance and Installing Node.js

In this lesson, you'll learn how to update your Amazon EC2 instance running Ubuntu and install Node.js version 22 along with npm. By the end of this lesson, you'll have a fully updated server ready for your Node.js applications. Key terms include EC2, Ubuntu, Node.js, npm, and SSH.

Step 1: Update Your EC2 Instance

To ensure your EC2 instance is up-to-date, you need to run a couple of commands in the terminal. First, connect to your EC2 instance via SSH. Once connected, execute the following commands:

#Create directory

App1:

```
ubuntu@ip-10-0-12-187:~$ mkdir app1
```

```
ubuntu@ip-10-0-12-187:~$ cd app1/
```

```
ubuntu@ip-10-0-12-187:~/app1$
```

App2:

```
ubuntu@ip-10-0-29-200:~$ mkdir app2
ubuntu@ip-10-0-29-200:~$ cd app2/
ubuntu@ip-10-0-29-200:~/app2$
```

#update and upgrade os version:

Note: The command `sudo apt update` refreshes the list of available packages and their versions, while `sudo apt upgrade -y` installs the latest versions of all installed packages. The `-y` flag automatically confirms the installation.

App1:

```
sudo apt update
sudo apt upgrade -y
```

App2:

```
sudo apt update
sudo apt upgrade -y
```

Step 2: Install Node.js 22 and npm

Node.js can be installed using the Node Source repository, which provides up-to-date versions of Node.js. Follow these steps to install Node.js 22 and npm:

App1:

```
curl -fsSL https://deb.nodesource.com/setup\_22.x | sudo -E bash -
sudo apt install -y nodejs
```

#version check

```
npm -v
node -v
```

```
ubuntu@ip-10-0-12-187:~/app1$ npm -v
10.9.8
ubuntu@ip-10-0-12-187:~/app1$ node -v
v22.23.1
ubuntu@ip-10-0-12-187:~/app1$
```

App2:

```
curl -fsSL https://deb.nodesource.com/setup\_22.x | sudo -E bash -
sudo apt install -y nodejs
```

#version check

```
npm -v
node -v
```

```
ubuntu@ip-10-0-29-200:~/app2$ npm -v
10.9.8
ubuntu@ip-10-0-29-200:~/app2$ node -v
v22.23.1
ubuntu@ip-10-0-29-200:~/app2$
```

Deploying a Node.js Application on EC2

In this lesson, you will learn how to deploy the TravelMemory application from the GitHub repository on an EC2 instance. This process includes setting up the Node.js backend, configuring environment variables, and testing the server. Key technologies covered include Node.js, Express, MongoDB, and AWS EC2.

Step 1: Cloning the Repository

First, you need to clone the GitHub repository to your EC2 instance. Use the following command in your terminal:

App1:

```
ubuntu@ip-10-0-12-187:~/app1$ git clone
https://github.com/UnpredictablePrashant/TravelMemory.git
Cloning into 'TravelMemory'...
remote: Enumerating objects: 122, done.
remote: Counting objects: 100% (73/73), done.
remote: Compressing objects: 100% (44/44), done.
remote: Total 122 (delta 35), reused 29 (delta 29), pack-reused 49 (from 1)
Receiving objects: 100% (122/122), 199.75 KiB | 6.89 MiB/s, done.
Resolving deltas: 100% (42/42), done.
ubuntu@ip-10-0-12-187:~/app1$
```

App2:

```
ubuntu@ip-10-0-29-200:~/app2$ git clone
https://github.com/UnpredictablePrashant/TravelMemory.git
Cloning into 'TravelMemory'...
remote: Enumerating objects: 122, done.
remote: Counting objects: 100% (73/73), done.
remote: Compressing objects: 100% (44/44), done.
remote: Total 122 (delta 35), reused 29 (delta 29), pack-reused 49 (from 1)
Receiving objects: 100% (122/122), 199.75 KiB | 7.40 MiB/s, done.
Resolving deltas: 100% (42/42), done.
ubuntu@ip-10-0-29-200:~/app2$
```

Step 2: Navigate to the Backend Folder

App1:

```
ubuntu@ip-10-0-12-187:~/app1$ cd TravelMemory/
ubuntu@ip-10-0-12-187:~/app1/TravelMemory$ ls -ltr
total 20
-rw-rw-r-- 1 ubuntu ubuntu 1075 Jun 29 10:21 LICENSE
-rw-rw-r-- 1 ubuntu ubuntu 442 Jun 29 10:21 azure-pipelines.yml
-rw-rw-r-- 1 ubuntu ubuntu 1446 Jun 29 10:21 README.md
drwxrwxr-x 5 ubuntu ubuntu 4096 Jun 29 10:21 backend
drwxrwxr-x 4 ubuntu ubuntu 4096 Jun 29 10:21 frontend
ubuntu@ip-10-0-12-187:~/app1/TravelMemory$
```

App2:

```
ubuntu@ip-10-0-29-200:~/app2$ cd TravelMemory/
ubuntu@ip-10-0-29-200:~/app2/TravelMemory$ ls -ltr
total 20
-rw-rw-r-- 1 ubuntu ubuntu 1075 Jun 29 10:21 LICENSE
-rw-rw-r-- 1 ubuntu ubuntu 442 Jun 29 10:21 azure-pipelines.yml
-rw-rw-r-- 1 ubuntu ubuntu 1446 Jun 29 10:21 README.md
```



```
drwxrwxr-x 5 ubuntu ubuntu 4096 Jun 29 10:21 backend
drwxrwxr-x 4 ubuntu ubuntu 4096 Jun 29 10:21 frontend
ubuntu@ip-10-0-29-200:~/app2/TravelMemory$
```

Step 3: Create the .env File

Create a .env file in the backend directory to store your environment variables. Use the command below to create and edit the file

App1:

```
ubuntu@ip-10-0-12-187:~/app1/TravelMemory/backend$ vi .env
```

```
ubuntu@ip-10-0-12-187:~/app1/TravelMemory/backend$ cat .env
MONGO_URI='mongodb+srv://travelmemory:system123
@cluster0.hfeojne.mongodb.net/travelmemory?retryWrites=true&w=majority'
PORT=3001
```

App2:

```
ubuntu@ip-10-0-29-200:~/app2/TravelMemory/backend$ vi .env
```

```
ubuntu@ip-10-0-29-200:~/app2/TravelMemory/backend$ cat .env
MONGO_URI='mongodb+srv://travelmemory:system123
@cluster0.hfeojne.mongodb.net/travelmemory?retryWrites=true&w=majority'
PORT=3001
```

Step 4: Install Necessary Packages

Next, you need to install the required packages. Run the following command to install the dependencies defined in package.json:

App1:

```
npm install
```

App2:

```
npm install
```

Step 5: Run the Backend Application

App1:

```
#run on nohup
nohup node index.js &
```

#Test the API Endpoint

```
ubuntu@ip-10-0-12-187:~/app1/TravelMemory/backend$ curl http://34.232.50.157:3001/hello
Hello World!
```

App2:

```
#run on nohup
nohup node index.js &
```

#Test the API Endpoint

```
ubuntu@ip-10-0-29-200:~/app2/TravelMemory/backend$ curl http://52.91.115.85:3001/hello
```

Hello World!

Step6: Navigate to Your Frontend Directory

Create the .env File:

In your frontend directory, create a `.env` file to store environment variables. This file will contain the backend URL that your React app will use to connect to the server.

App1:

```
# Change the directory to frontend  
Cd /app1/TravelMemory/frontend
```

```
#create .env file  
echo "REACT_APP_BACKEND_URL=http://34.232.50.157:3001" > .env
```

```
#check  
ubuntu@ip-10-0-12-187:~/app1/TravelMemory/frontend$ cat .env  
REACT_APP_BACKEND_URL=http://34.232.50.157:3001
```

App2:

```
# Change the directory to frontend  
Cd /app2/TravelMemory/frontend
```

```
#create .env file  
echo "REACT_APP_BACKEND_URL=http://52.91.115.85:3001" > .env
```

```
#check  
ubuntu@ip-10-0-29-200:~/app2/TravelMemory/frontend$ cat .env  
REACT_APP_BACKEND_URL=http://52.91.115.85:3001
```

Step 7: Install Required Packages

Now that your environment variables are set, you need to install the required packages for your application. Run the following command:

App1:

```
npm install
```

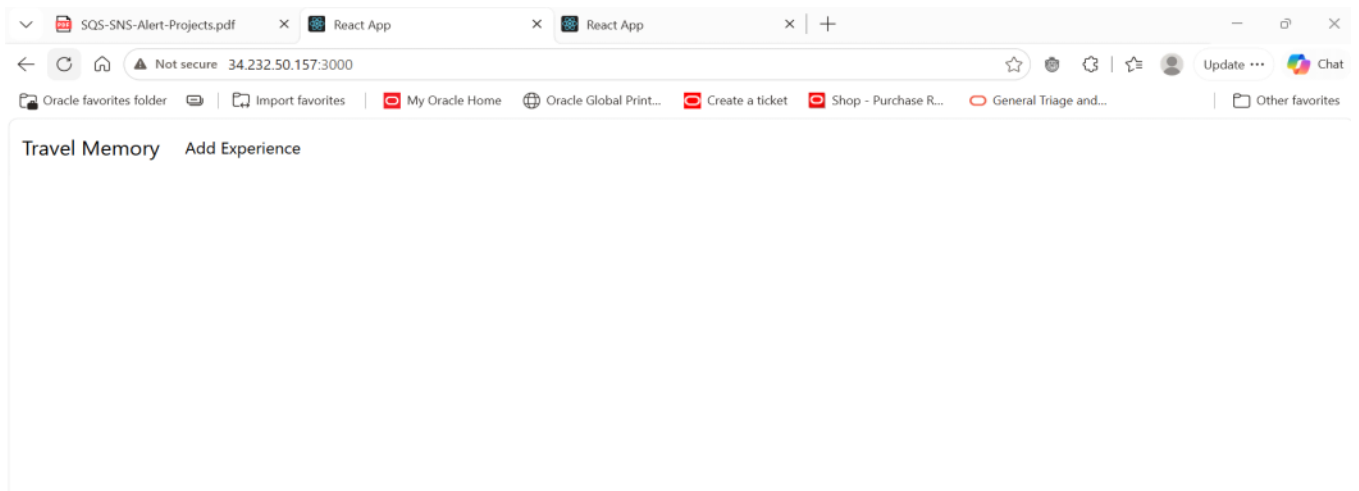
App2:

```
npm install
```

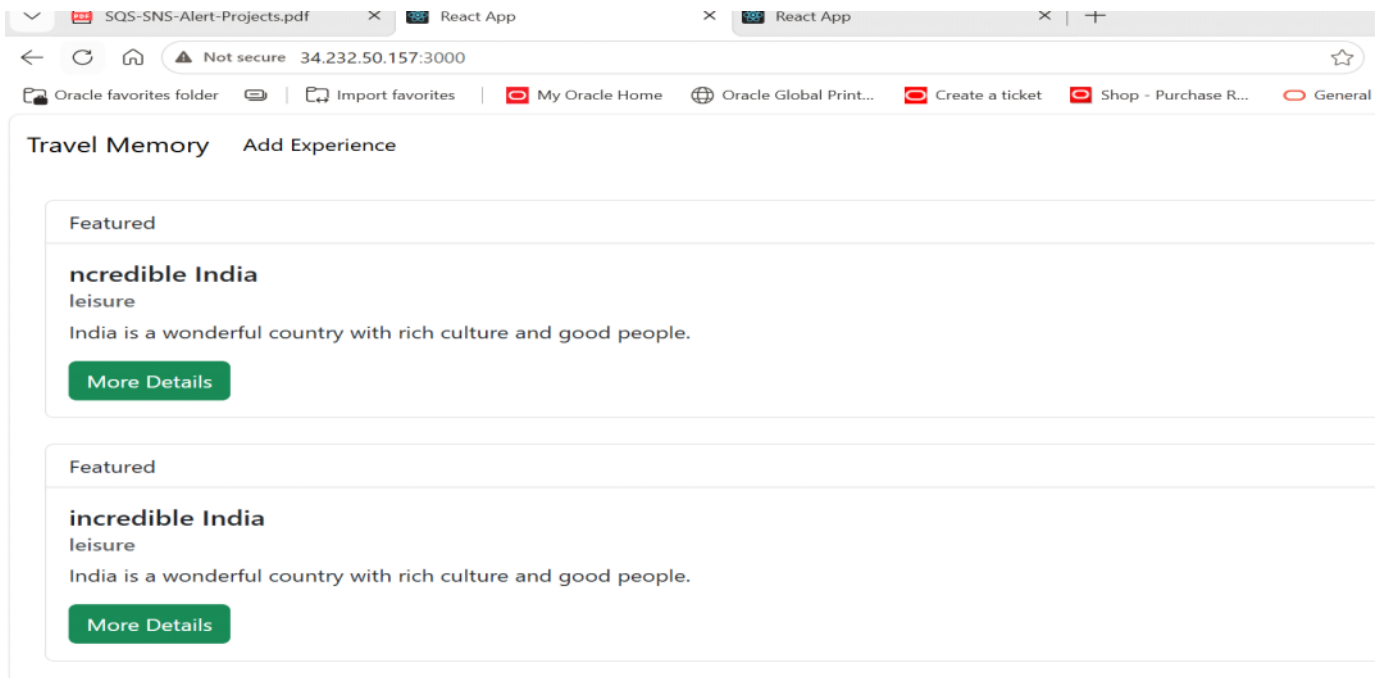
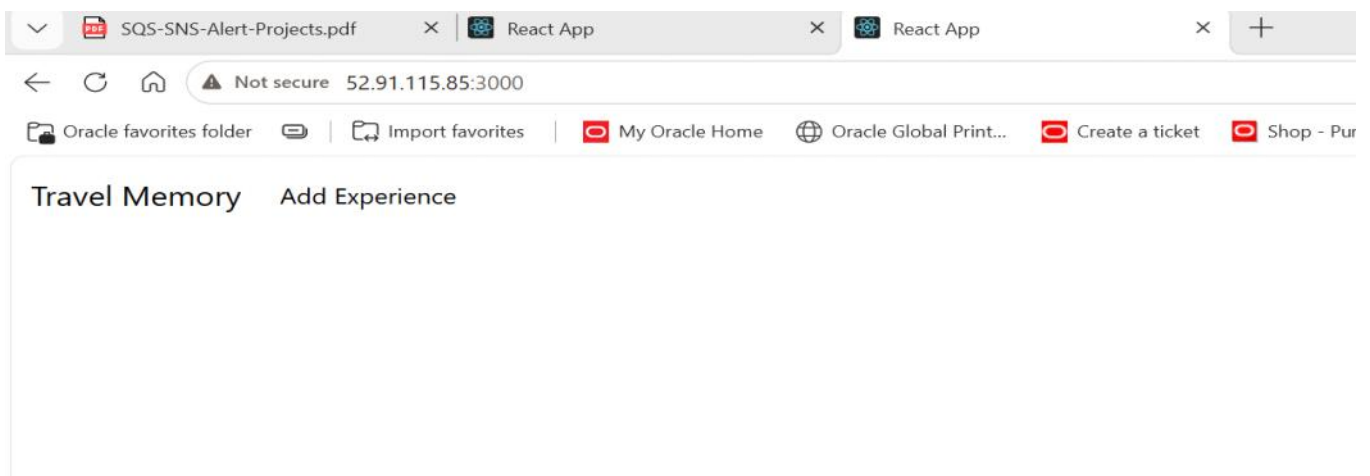
Step 8: Run Your Application

After the packages are installed, you can run your application using the command below. This will start your React application on port 3000.

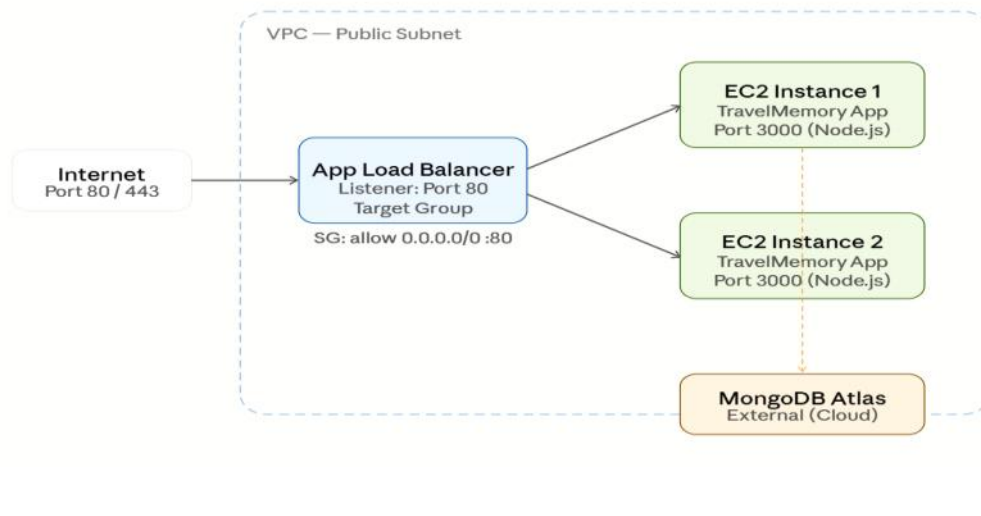
App1:



App2:



Step-by-step: ALB setup for TravelMemory on 2 EC2 instances



Step 1 — Verify your app is running on both EC2s

SSH into each EC2 and confirm the app is listening:

App1:

#Test the API Endpoint

```
ubuntu@ip-10-0-12-187:~/app1/TravelMemory/backend$ curl http://34.232.50.157:3001/hello
Hello World!
```

App2:

#Test the API Endpoint

```
ubuntu@ip-10-0-29-200:~/app2/TravelMemory/backend$ curl http://52.91.115.85:3001/hello
Hello World!
```

Step 2: Create a Target Group

Go to **EC2** → **Target Groups** → **Create target group**

Fill in:

Field	Value
Target type	Instances
Target group name	travelmemory-tg
Protocol	HTTP
Port	3000 (your app port)
VPC	Select your VPC

Health check settings:

- Protocol: HTTP
- Path: / (or /health if your app has a health endpoint)
- Healthy threshold: 2
- Interval: 30 seconds

Click **Next**, then select **both EC2 instances** → click **Include as pending below**

Click Create target group

EC2

Target groups

Create target group

Step 1

Create target group

Step 2 - recommended

Register targets

Step 3

Review and create

Create target group

A target group can be made up of one or more targets. Your load balancer routes requests to the targets in a target group and performs health checks on the targets.

Settings - immutable

Choose a target type and the load balancer and listener will route traffic to your target. These settings can't be modified after target group creation.

Target type

Indicate what resource type you want to target. Only the selected resource type can be registered to this target group.

Instances

Supports load balancing to instances in a VPC. Integrate with Auto Scaling Groups or ECS services for automatic management.

Suitable for: ALB NLB GWLB

IP addresses

Supports load balancing to VPC and on-premises resources. Facilitates routing to IP addresses and network interfaces on the same instance. Supports IPv6 targets.

Suitable for: ALB NLB GWLB

Lambda function

Supports load balancing to a single Lambda function. ALB required as traffic source.

Suitable for: ALB

Application Load Balancer

Allows use of static IP addresses and PrivateLink with an Application Load Balancer. NLB required as traffic source.

Suitable for: NLB

Target group name

Name must be unique per Region per AWS account.

travelmemory-tg

Accepts: a-z, A-Z, 0-9, and hyphen (-). Can't begin or end with hyphen. 1-32 total characters; Count: 15/32

Protocol

Protocol for communication between the load balancer and targets.

HTTP

Port

Port number where targets receive traffic. Can be overridden for individual targets during registration.

3000

1-65535

1-65535

IP address type

Only targets with the indicated IP address type can be registered to this target group.

IPv4

Each instance has a default network interface (eth0) that is assigned the primary private IPv4 address. The instance's primary private IPv4 address is the one that will be applied to the target.

IPv6

Each instance you register must have an assigned primary IPv6 address. This is configured on the instance's default network interface (eth0). [Learn more](#)

VPC

Select the VPC with the instances that you want to include in the target group. Only VPCs that support the IP address type selected above are available in this list.

vpc-0f6b9f93a5304573b (travl-mem-vpc-vpc)

10.0.0.0/16

Create VPC

Protocol version

HTTP1

Send requests to targets using HTTP/1.1. Supported when the request protocol is HTTP/1.1 or HTTP/2.

HTTP2

Send requests to targets using HTTP/2. Supported when the request protocol is HTTP/2 or gRPC, but gRPC-specific features are not available.

gRPC

Send requests to targets using gRPC. Supported when the request protocol is gRPC.

Health checks

The associated load balancer periodically sends requests, per the settings below, to the registered targets to test their status.

Health check protocol

Health checks

The associated load balancer periodically sends requests, per the settings below, to the registered targets to test their status.

Health check protocol

HTTP

Health check path

Use the default path of "/" to perform health checks on the root, or specify a custom path if preferred.

/

Up to 1024 characters allowed.

Advanced health check settings

Target optimizer - optional

Enforce a maximum number of concurrent requests on a target. This is particularly useful for compute-intensive workloads like complex data processing or inference.

Target optimizer

Off - Default

Use standard load balancing.

On

Enforce a maximum number of concurrent requests on a target. Requires targets to have target optimizer agent installed.

Attributes

Certain default attributes will be applied to your target group. You can view and edit them after creating the target group.

Step 1

● Create target group

Step 2 - recommended

● Register targets

Step 3

○ Review and create

Register targets - recommended

This is an optional step to create a target group. However, to ensure that your load balancer routes traffic to this target group you must register your targets.

Available instances (2)

Filter instances

< 1 >

☐	Instance ID	Name	State	Security groups	Zone	Private IPv4 address	Subnet
☐	i-03b776101143b7c	app2	Running	travl-mem-sg	us-east-1b	10.0.29.200	subn
☐	i-03985dde878f093a2	app1	Running	travl-mem-sg	us-east-1a	10.0.12.187	subn

0 selected

Ports for the selected instances

Ports for routing traffic to the selected instances.

3000

1-65535 (separate multiple ports with commas)

Include as pending below

0 pending

Cancel

Previous

Next

Step 1

● Create target group

Step 2 - recommended

● Register targets

Step 3

● Review and create

Review and create

Review your target group configuration before creating

Step 1: Target group details

Edit

Target group details

Name	travlmemory-tg	Target type	Instance	Protocol : Port	HTTP: 3000	Protocol version	HTTP1
VPC	vpc-0f6b9f93a5304573b						
		IP address type	IPv4				

Health check details

Health check protocol	HTTP	Health check path	/	Health check port	traffic-port	Interval	30 seconds
Timeout	5 seconds	Healthy threshold	2	Unhealthy threshold	2	Success codes	200

Health check details

Health check protocol HTTP	Health check path /	Health check port traffic-port	Interval 30 seconds
Timeout 5 seconds	Healthy threshold 2	Unhealthy threshold 2	Success codes 200

Step 2: Register targets

Edit

Targets (0)

Instance ID	Name	Port	Zone
No targets added			

Server-side tasks and status

After completing and submitting the above steps, all server-side tasks and their statuses become available for monitoring.

Cancel

Previous

Create target group

Step 3 — Create the Application Load Balancer

1. Go to **EC2** → **Load Balancers** → **Create Load Balancer**

2. Choose **Application Load Balancer**

3. **Basic configuration:**

Field	Value
Name	travelmemory-alb
Scheme	Internet-facing
IP address type	IPv4

4. **Network mapping:**

- VPC: select your VPC
- Availability Zones: select **both AZs** where your EC2s live, and pick the **public subnets**

5. **Security groups:**

- Create a new SG or use an existing one
- It must allow **Inbound HTTP port 80** from 0.0.0.0/0

6. **Listeners and routing:**

- Protocol: HTTP, Port: 80
- Default action: Forward to → travelmemory-tg (the target group you just created)

7. Click **Create load balancer**

EC2 > Load balancers > Create Application Load Balancer

Create Application Load Balancer

The Application Load Balancer distributes incoming HTTP and HTTPS traffic across multiple targets such as Amazon EC2 instances, microservices, and containers, based on request attributes. When the load balancer receives a connection request, it evaluates the listener rules in priority order to determine which rule to apply, and if applicable, it selects a target from the target group for the rule action.

How Application Load Balancers work

Basic configuration

Load balancer name

Name must be unique within your AWS account and can't be changed after the load balancer is created.

travelmemory-alb

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Scheme

Scheme can't be changed after the load balancer is created.

Internet-facing

Serves internet-facing traffic.

Has public IP addresses.

DNS name resolves to public IPs.

Requires a public subnet.

Internal

Serves internal traffic.

Has private IP addresses.

DNS name resolves to private IPs.

Compatible with the IPv4 and Dualstack IP address types.

Load balancer IP address type

Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types. Public IPv4 addresses have an additional cost.

IPv4

Includes only IPv4 addresses.

Dualstack

Includes IPv4 and IPv6 addresses.

Dualstack without public IPv4

Includes a public IPv6 address, and private IPv4 and IPv6 addresses. Compatible with Internet-facing load balancers only.

Travel-memory-ec2-LB Page 15

Network mapping [info](#)

The load balancer routes traffic to targets in the selected subnets, and in accordance with your IP address settings.

VPC [Info](#)

The load balancer will exist and scale within the selected VPC. The selected VPC is also where the load balancer targets must be hosted unless routing to Lambda or on-premises targets, or if using VPC peering. To confirm the VPC for your targets, view [target groups](#).

vpc-0f6b9f93a5304573b (travl-mem-vpc-vpc)
10.0.0.0/16



Create VPC [↗](#)

IP pools [Info](#)

You can optionally choose to configure an IPAM pool as the preferred source for your load balancers IP addresses. Create or view Pools in the [Amazon VPC IP Address Manager console](#).

☐ Use IPAM pool for public IPv4 addresses

The IPAM pool you choose will be the preferred source of public IPv4 addresses. If the pool is depleted IPv4 addresses will be assigned by AWS.

Availability Zones and subnets [Info](#)

Select at least two Availability Zones and a subnet for each zone. A load balancer node will be placed in each selected zone and will automatically scale in response to traffic. The load balancer routes traffic to targets in the selected Availability Zones only.

☒ us-east-1a (use1-az1)

Subnet

Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently. Selected subnets must have Network ACL inbound and outbound ALLOW rules, and internet gateway (IGW) routes configured to allow traffic.

subnet-04953f1fbcf97206d (travl-mem-vpc-subnet-public1-us-east-1a)
10.0.0.0/20
Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (✓)

☒ us-east-1b (use1-az2)

Subnet

Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently. Selected subnets must have Network ACL inbound and outbound ALLOW rules, and internet gateway (IGW) routes configured to allow traffic.

subnet-06a35e06ce20ad88f (travl-mem-vpc-subnet-public2-us-east-1b)
10.0.16.0/20
Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (✓)

subnet-06a35e06ce20ad88f (travl-mem-vpc-subnet-public2-us-east-1b)
10.0.16.0/20
Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (✓)

Security groups [Info](#)

A security group is a set of firewall rules that control the traffic to your load balancer. Listed security groups are in the same VPC as you selected above. Selected security groups must include at least 1 inbound rule and 1 outbound rule to allow traffic to your load balancer.

Security groups

Select up to 5 security groups



Create security group [↗](#)

travl-mem-sg-0d17f8c3465821ea2 [✕](#)
Inbound rule (4) Outbound rule (1)

Listeners and routing [Info](#)

A listener is a process that checks for connection requests using the port and protocol you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets.

▼ Listener HTTP:80

Remove

Protocol

HTTP

Port

80

1-65535

Default action [Info](#)

The default action is used if no other rules apply. Choose the default action for traffic on this listener.

Routing action

☒ Forward to target groups

☐ Redirect to URL

☐ Return fixed response

Forward to target group [Info](#)

Choose a target group and specify routing weight or [create target group](#).

Target group

travlmemory-tg

Target type: Instance, IPv4 | Target stickiness: Off

HTTP



Weight

1

0-999

Percent

100%

+ Add target group

You can add up to 4 more target groups.

Target group stickiness [Info](#)

Enables the load balancer to bind a user's session to a specific target group. To use stickiness the client must support cookies. If you want to bind a user's session to a specific target, turn on the Target Group attribute Stickiness.

☐ Turn on target group stickiness

EC2 > Load balancers > travmemory-alb

EC2

- Dashboard
- AWS Global View
- Events
- Instances
 - Instances
 - Instance Types
 - Launch Templates
 - Spot Requests
 - Savings Plans
 - Reserved Instances
 - Dedicated Hosts
 - Capacity Reservations
 - Capacity Manager
- Images
 - AMIs
 - AMI Catalog
- Elastic Block Store
 - Volumes
 - Snapshots
 - Lifecycle Manager
- Network & Security
 - Security Groups
 - Elastic IPs
 - Placement Groups
 - Key Pairs

Details

Load balancer type Application	Status Provisioning	VPC vpc-0f6b9f93a5304573b	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone Z35SXDOTRQ7X7K	Availability Zones subnet-06a35e06ce20ad88f us-east-1b (use1-az2) subnet-04953f1fbc97206d us-east-1a (use1-az1)	Date created June 29, 2026, 17:09 (UTC+05:30)
Load balancer ARN arn:aws:elasticloadbalancing:us-east-1:675134942081:loadbalancer/app/travmemory-alb/220d7556fb016d5a		DNS name info travmemory-alb-1556311436.us-east-1.elb.amazonaws.com (A Record)	

Listeners and rules | Network mapping | Resource map | Security | Monitoring | Integrations | Attributes | Capacity | Tags

Listeners and rules (1)

A listener checks for connection requests on its configured protocol and port. Incoming requests are evaluated against rules in priority order; the first matching rule determines routing. If no rule matches, the default action is applied.

Protocol:Port	Default action	Rules	ARN	Security policy	Default SSL/TLS certificate	mTLS	Trust store
HTTP:80	Forward to target group travmemory-tg: 1 (100%) Target group stickiness: Off	1 rule	ARN	Not applicable	Not applicable	Not applicable	Not applicable

Lock down EC2 Security Groups (important)

Now go back and update the EC2 SG inbound rule you set in Step 2:

- Source: change from 0.0.0.0/0 to the **ALB's Security Group ID** (e.g. sg-0abc123...)
- This ensures your app is only reachable through the ALB, not directly via EC2 IP

EC2 → Target Groups → travmemory-tg → Targets tab — both should show status **Healthy**

EC2 > Target groups

Spot requests

- Savings Plans
- Reserved Instances
- Dedicated Hosts
- Capacity Reservations
- Capacity Manager

Target groups (1/1)

Name	ARN	Port	Protocol	Target type	Load balancer	VPC ID
travmemory-tg	arn:aws:elasticloadbalancing:us-east-1:675134942081:targetgroup/travmemory-tg/220d7556fb016d5a	3000	HTTP	Instance	None associated	vpc-0f6b9f93a5304573b

Target group: travmemory-tg

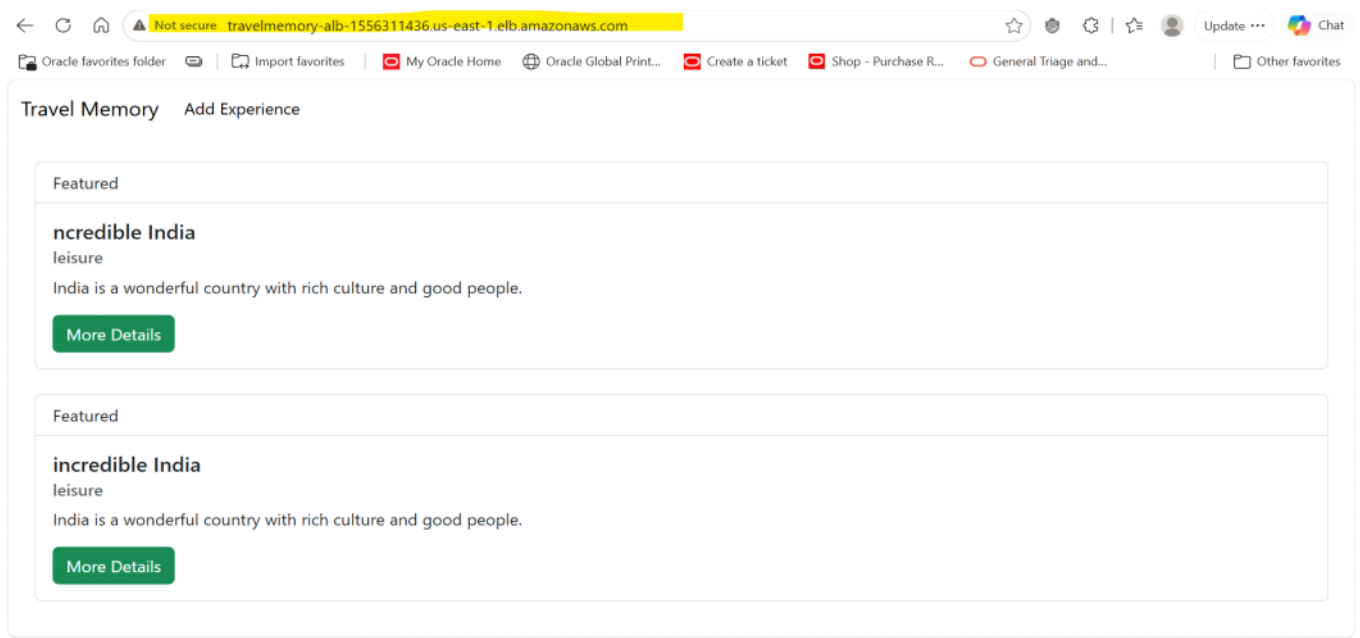
IP address type IPv4	Load balancer None associated
2 Total targets	2 Healthy 0 Anomalous 0 Unhealthy 0 Unused 0 Initial 0 Draining

► Distribution of targets by Availability Zone (AZ)
Select values in this table to see corresponding filters applied to the Registered targets table below.

Test the setup

- Go to **Load Balancers** and copy the **DNS name** of your ALB (looks like travmemory-alb-123456.ap-south-1.elb.amazonaws.com)
- Open it in a browser: <http://<alb-dns-name>>
- Your TravelMemory app should load

DNS URL : <http://travmemory-alb-1556311436.us-east-1.elb.amazonaws.com>



Note Done below one on this projects

(Optional but recommended) Point a domain to the ALB

If you have a domain, create a **CNAME record** in Route 53 (or your DNS provider) pointing to the ALB DNS name. Do **not** use an A record — ALB IPs change.